Travaux pratiques n°1 — #Classes et Objets en Python

Matière : Programmation Avancée pour la Data Science

Discipline : 2ère année Science de Données et Intelligence Artificielle (SDIA)

Année Universitaire : 2026-2027 / S1

Enseignants : Fatma Ben Said
Sihem Loukil

Problème

Une équipe de Data Scientists souhaite développer une application Python pour gérer et analyser des données provenant de capteurs intelligents.
Les capteurs enregistrent régulièrement des observations. Chaque *observation* contient un *identifiant*, le *nom du capteur*, une *température* et une *humidité*. Votre objectif est de concevoir une petite application orientée objet permettant de **créer, gérer, valider et analyser un ensemble d'observations**.

Travail à Faire :

1. Créer la classe **`Observation`** afin de représenter une observation collectée par un capteur. La classe possède les **attributs d'instance** suivants : `id` (identifiant de l'observation), `capteur` (nom du capteur), `temperature` (température mesurée) et `humidite` (humidité mesurée).

Elle possède également un **attribut de classe** `nombre_observations`, permettant de mémoriser le nombre total d'objets `Observation` créés. Cet attribut est initialisé à 0.

Ajouter un **constructeur** permettant d'initialiser les attributs d'une observation et d'incrémenter `nombre_observations` à chaque création d'un nouvel objet.

2. Ajouter la méthode `est_valide()` afin de vérifier si les valeurs mesurées sont valides. Une observation est considérée comme valide si :

- `-20 <= temperature <= 60` et
- `0 <= humidite <= 100`

La méthode retourne `True` si les deux conditions sont respectées et `False` sinon.

3. Redéfinir la méthode `__str__()` afin d'obtenir une représentation lisible d'une observation comme suit :

```
Observation #12 | Capteur : C01 | Température : 24.5°C | Humidité : 51%
```

4. Créer une classe `JeuDonnees` afin de regrouper plusieurs objets `Observation` et de faciliter leur gestion. La classe possède **un attribut privé** `observations` contenant la liste des observations enregistrées.

5. Ajouter à la classe `JeuDonnees` les méthodes suivantes :

- `ajouter_observation(obs)` : Cette méthode reçoit `obs` un objet de type `Observation` et doit :
  - ✓ ajouter l'objet au jeu de données **uniquement s'il s'agit d'une instance de la classe** `Observation` ;
  - ✓ si l'objet fourni n'est pas une `Observation`, ne pas l'ajouter et afficher le message : **"Erreur : l'objet fourni n'est pas une Observation."**
- `ajouter_plusieurs(*observations)` : Cette méthode reçoit un nombre variable d'objets `Observation` et doit :
  - ✓ parcourir les observations reçues ;
  - ✓ utiliser la méthode **ajouter_observation()** pour ajouter chaque observation au jeu de données.
- `supprimer_invalides()` : Cette méthode permet de supprimer du jeu de données toutes les observations invalides. Elle doit parcourir les observations du jeu de données et conserver uniquement les observations pour lesquelles la méthode `est_valide()` retourne `True`.

6. Ajouter les méthodes d'accès aux données afin de respecter le principe d'encapsulation :

- `get_observations()` : retourne les observations enregistrées ;
- `set_observations(observations)` : remplace les observations enregistrées par une nouvelle liste après avoir vérifié que les éléments fournis sont bien des objets `Observation`. Si ce n'est pas le cas, ne pas modifier la liste et afficher le message : **"Erreur : tous les éléments doivent être des objets Observation."**

7. **Ajouter à la classe `JeuDonnees` la méthode** `filtrer(condition)` afin de retourner les observations qui vérifient une condition donnée sous forme de **liste**. La méthode reçoit `condition`, une fonction en paramètre qui sera appliquée à chaque observation du jeu de données et utilise `filter()` pour effectuer le filtrage.

Exemple d'utilisation : filtrer le jeu de données afin d'obtenir uniquement les observations dont la température est supérieure à **25°C**.

8. **Ajouter la méthode** `extraire(fct)` afin d'appliquer une fonction `fct` à chaque observation et de retourner les résultats obtenus (utiliser une fonction pour **transformer chaque observation**).
La méthode doit utiliser `map()` et retourner l'ensemble des résultats sous forme de liste. *Exemple : jeu.extraire(...) retourne la liste des températures des observations → transformer chaque observation en sa température.*

9. Ajouter à la classe **`JeuDonnees`** la méthode `trier(critere)` afin de retourner les observations **triées selon un critère fourni sous forme de fonction** (`critere`).
Utiliser `sorted()` et une fonction `lambda`.
Par exemple, permettre de trier les observations selon leur température croissante.

10. Écrire une fonction `analyser(observations, operation)` permettant d'appliquer une opération donnée à une liste d'observations. La fonction reçoit une liste d'objets `Observation` et une fonction `operation`, puis retourne le résultat de cette opération. Tester cette fonction avec différentes opérations, par exemple pour obtenir :

- *la température moyenne*
- *la température maximale*
- *l'humidité moyenne*

11. La classe **`Observation`** a été initialement développée de manière classique. On souhaite maintenant simplifier sa définition.
Transformer la classe **`Observation`** en une `dataclass` en utilisant les annotations de type appropriées pour ses attributs. La nouvelle définition doit conserver les mêmes informations et permettre de créer les objets **`Observation`** comme précédemment.

12. Adapter la classe **`Observation`** après l'utilisation de `@dataclass`. Identifier les éléments qui ne sont plus nécessaires grâce à la génération automatique du constructeur et conserver les méthodes nécessaires au fonctionnement de l'application, notamment `est_valide()` et `__str__()`. Conserver également le comptage des objets avec `nombre_observations`.

13. Créer quelques objets `Observation` à partir de la nouvelle version `dataclass` et vérifier que leur création et leurs méthodes fonctionnent correctement.

14. Écrire le programme principal permettant de tester l'ensemble des fonctionnalités développées. Le programme devra :

a. Créer quatre observations nommées `obs1, obs2, obs3 et obs4`, avec les valeurs suivantes : **["C01", 24.5, 51] , ["C02", 72, 45], ["C03", 18.2, 65], ["C04", 30.7, 80]**

b. Afficher les quatre observations et le nombre total d'observations créées.

c. Modifier la température à `22.5` et l'humidité à `55` de `obs2`, puis vérifier sa validité.

d. Créer un objet **`JeuDonnees`** nommé `jeu.`

e. Ajouter `obs1, obs2, obs3 et obs4` au jeu de données `jeu`.

f. Afficher les observations enregistrées dans `jeu` sans accéder directement à l'attribut privé `observations`.

g. Supprimer les observations invalides.

h. Afficher la température moyenne et l'observation ayant la température maximale.

i. Fixer le *seuil de température* à **25°C** et afficher les observations dont la température est *supérieure à ce seuil*.

j. À partir des observations enregistrées dans `jeu`, construire et afficher une **liste contenant uniquement les températures des observations**.

NB. Utiliser `extraire()` pour obtenir la liste des températures

k. **Trier** les observations enregistrées dans `jeu` par **ordre croissant de température**, puis afficher les observations triées.

l. Utiliser la fonction `analyser()` pour calculer et afficher **la température moyenne des observations** et **l'humidité moyenne des observations**.

m. Modifier la température de `obs1` à **45°C**, puis afficher `obs1` avant et après la modification. Recalculer ensuite la **température moyenne** des observations et afficher le nouveau résultat.

Bon Travail 😊